Contributing to Stan

A Developer's Guide to the Core, Interfaces, and First Contributions

Florence Bockting

2026-08-13

Table of contents

Goals

This material is for anyone interested in contributing to Stan but unsure where to start. By the end of the tutorial, you will be able to:

- **Place yourself in the ecosystem**: Know which projects make up Stan, from the compiler and math library through `CmdStan` to the language interfaces and workflow packages, and work out which repository a given contribution belongs in.
- **Pick a contribution that fits you**: Recognise the full range of ways to help, from answering questions on Stan Discourse, to filing bug reports and feature requests, improving documentation, writing case studies, and contributing code.
- **Reach out effectively**: Open a GitHub issue or pull request that maintainers can act on, and know when Discourse is the better place to start the conversation instead.
- **Find the right information**: Match a question to the right resource among user guides, API documentation, case studies, and articles.
- **Follow a concrete first contribution**: The checklist walks step by step through the practicalities. It currently focuses on the R packages in the Stan ecosystem, though most of the workflow carries over to the other interfaces.

You do not need to be a professional software developer or statistician, and you do not need prior open-source experience.

1 What is Stan?

By the end of this section, you'll understand that Stan is a probabilistic programming language for specifying statistical models, an open-source community, as well as a broader software ecosystem.

1.1 Language, Community, Ecosystem

Stan is ...

- an open-source community consisting of people from a variety of different backgrounds (Cognitive Science, Computer Science, Statistics, etc.).
- an open-source ecosystem consisting of a core library, interfaces for multiple programming languages, and a growing collection of contributed packages.
- a probabilistic programming language for specifying statistical models.

1.1.1 An open-source community

We communicate via:

- Slack
  - different channels (e.g., #general, #help)
  - mainly developer discussions
- Discourse
  - questions, discussion, and announcements related to Stan
  - for both users and developers
- GitHub
  - code repositories
  - issue tracker for reporting bugs and requesting features
  - pull requests for contributing code, documentation, examples, etc.

- Stan Website
  - documentation, tutorials, case studies, etc.

1.1.2 An open-source ecosystem

Multiple packages/libraries that work together to provide a complete probabilistic programming environment

More on this later!

1.1.3 A probabilistic programming language

A probabilistic programming language for specifying statistical models.

```
data {
  int N;   // number of observations
  int J;   // number of students
  int K;   // number of items on exam
  array[N] int<lower=0, upper=J> student;
  array[N] int<lower=0, upper=K> item;
  array[N] int<lower=0, upper=1> y;
  vector[J] x;
  real mu_mu_a, mu_mu_b;
  real<lower=0> sigma_mu_a, sigma_mu_b, mu_sigma_a, mu_sigma_b;
}
transformed data {
  vector[J] x_adj = (x - mean(x))/sd(x);
}
parameters {
  real mu_a, mu_b;
```

```
  real<lower=0> sigma_a, sigma_b;
  vector<offset=mu_a, multiplier=sigma_a>[K] a;
  vector<offset=mu_b, multiplier=sigma_b>[K] b;
}
model {
  a ~ normal(mu_a, sigma_a);
  b ~ normal(mu_b, sigma_b);
  mu_a ~ normal(mu_mu_a, sigma_mu_a);
  mu_b ~ normal(mu_mu_b, sigma_mu_b);
  sigma_a ~ exponential(1/mu_sigma_a);
  sigma_b ~ exponential(1/mu_sigma_b);
  y ~ bernoulli(0.25 + 0.75*inv_logit(a[item] + b[item] .* x_adj[student]));
}
```

2 The Stan Ecosystem

By the end of this section, you'll understand that the Stan ecosystem is a stack of related projects mainly developed under stan-dev, ranging from the compiler and math library to `CmdStan`, language interfaces, and workflow packages. You'll be able to place a typical modeling task within that stack and identify which layer each piece belongs to.

2.1 A minimal Stan workflow

We start our workflow by writing a minimal Stan model in a `.stan` file.

Listing 2.1 `R · Stan model`

```
stan_code <- "
data {
  int<lower=0> N;
  vector[N] y;
}
parameters {
  real mu;
  real<lower=0> sigma;
}
model {
  mu ~ normal(0, 10);      // prior
  sigma ~ exponential(1);  // prior
  target += normal_lpdf(y | mu, sigma);  // likelihood
}
generated quantities {
  vector[N] log_lik;
  for (n in 1:N) {
    log_lik[n] = normal_lpdf(y[n] | mu, sigma);
  }
}
"
writeLines(stan_code, "model.stan")
```

Additionally, we provide some data for the input and save it as a JSON file.

Listing 2.2 `R · data`

```
library(cmdstanr)

stan_data <- list(N = 20, y = rnorm(20, mean = 5, sd = 2))
cmdstanr::write_stan_json(stan_data, "data.json")

> stan_data
$N
[1] 20

$y
 [1] 4.543589 4.899750 4.081620 8.698614 4.827003 2.903813 8.659615 5.165155
 [9] 5.148127 1.206675 2.368544 7.873981 5.071258 5.322390 6.913431 7.336083
[17] 3.462279 3.717310 1.620141 3.577615
```

2.1.1 Language interfaces

Next we can compile and sample from the Stan model. To do this, we use an interface, such as `cmdstanr` for R or `cmdstanpy` for Python, that wraps `CmdStan`.

`CmdStan` invokes

- `stanc3` to transpile the Stan model into a C++ file, which is then compiled and linked against
- the `math` library (implementing built-in functions for distributions, linear algebra, ODE solvers, etc., along with automatic differentiation) and
- the `stan` library (implementing the inference algorithms, such as HMC/NUTS).

The result is a standalone executable that the interface then runs to produce posterior samples.

Listing 2.3 `R · cmdstanr`

```r
library(cmdstanr)
library(bayesplot)
library(loo)

# compile Stan model
mod <- cmdstanr::cmdstan_model("model.stan")

# sample from the model
fit <- mod$sample(data = "data.json", chains = 4, iter_sampling = 1000)

# get draws
draws <- fit$draws(variables = c("mu", "sigma"))

# plotting and cross-validation
bayesplot::mcmc_trace(draws)
loo::loo(fit$draws("log_lik"))
```

2.1.1.1 The R interface `cmdstanr`

Compile and sample from the Stan model in R with `cmdstanr`. Use the fitted model to extract posterior draws, plot MCMC trace plots with `bayesplot`, and run approximate leave-one-out cross-validation with `loo`.

2.1.1.2 The Python interface `cmdstanpy`

Compile and sample from the Stan model in Python with `cmdstanpy`. Use the fitted model to extract posterior draws, plot MCMC trace plots, and run approximate leave-one-out cross-validation using `arviz`.

2.1.1.3 The Command-line interface `CmdStan`

`cmdstanr` and `cmdstanpy` are thin language wrappers around `CmdStan`, the command-line interface to Stan. `CmdStan` itself invokes `stanc3` to compile a `.stan` file into a standalone executable, which can be run directly from the command line to sample from (or optimize, etc.) the model.

Listing 2.4 Python · cmdstanpy

```python
from cmdstanpy import CmdStanModel
import arviz as az

# compile Stan model
mod = CmdStanModel(stan_file="model.stan")

# sample from the model
fit = mod.sample(data="data.json", chains=4, iter_sampling=1000)

# get draws
idata = az.from_cmdstanpy(fit, log_likelihood="log_lik")

# plotting and cross-validation
az.plot_trace(idata, var_names=["mu", "sigma"])
az.loo(idata)
```

Listing 2.5 Bash

```bash
# run from the CmdStan directory
make model

./model sample num_chains=4 num_samples=1000 \
  data file=data.json \
  output file=output.csv

$CMDSTAN/bin/stansummary output_*.csv
$CMDSTAN/bin/diagnose output_*.csv
```

2.1.2 Higher-level modelling frameworks

Instead of writing a Stan model from scratch, you can use higher-level modelling frameworks that let you specify your model using formula syntax. Two example frameworks are

- brms, which dynamically generates and compiles custom Stan code for your specified model, and
- rstanarm, which fits your model using a set of pre-compiled Stan programs for common model families (avoiding compilation time, at the cost of some flexibility).

Listing 2.6 `R · brms`

```r
library(brms)

dat <- data.frame(y = jsonlite::fromJSON("data.json")$y)

fit <- brms::brm(
  formula = y ~ 1,
  data = dat,
  family = gaussian(),
  prior = c(
    prior(normal(0, 10), class = Intercept),
    prior(exponential(1), class = sigma)
  ),
  chains = 4,
  iter = 2000
)
```

2.2 The Stan ecosystem and its libraries

To summarize what we have seen so far, we can outline the different layers in the Stan ecosystem and how they relate to one another.

Different layers in the Stan ecosystem

- **Layer 1: Compiler & math**: `stanc3` · `stan` · `math`
  - Contribute to the fundamental building blocks of the Stan language: compiler transpilation, autodiff, or core inference algorithms.
- **Layer 2: Command-line engine**: `cmdstan`
  - Contribute to the command-line interface that drives compilation and execution.
- **Layer 3: Language interfaces**: `cmdstanr` · `cmdstanpy` · `rstan` · …
  - Contribute to environment-specific host language wrappers (R, Python, Julia, etc.) that interface with Stan.
- **Layer 4: Workflow & analysis**: `posterior` · `bayesplot` · `loo` · `priorsense` · …
  - Contribute to downstream packages used among others for visualization, posterior diagnostics, cross-validation, and sensitivity analysis.

Listing 2.7 `R · rstanarm`

```r
library(rstanarm)

dat <- data.frame(y = jsonlite::fromJSON("data.json")$y)

fit <- rstanarm::stan_glm(
  formula = y ~ 1,
  data = dat,
  family = gaussian(),
  prior_intercept = normal(0, 10, autoscale = FALSE),
  prior_aux = exponential(1, autoscale = FALSE),
  chains = 4,
  iter = 2000
)
```

2.2.1 Compiler, math, and CmdStan

Package	Description	Relations	Language
math	Autodiff library for distributions, linear algebra, and gradients	Used by generated model code	C++
stanc3	Compiler that translates `.stan` to C++	Feeds `cmdstan` and `rstan`	OCaml
stan	Inference algorithms and services (HMC/NUTS, etc.)	Built on `math`; used via `cmdstan` / `rstan`	C++
cmdstan	Command-line engine to compile and run Stan models	Wraps `stanc3` + `stan` + `math`; backend for `cmdstanr` / `cmdstanpy`	C++ / CLI

2.2.2 Language interfaces

Package	Description	Relations	Language
cmdstanr	R wrapper around CmdStan; returns `CmdStanMCMC`	Calls `cmdstan`; used with `posterior`, `bayesplot`, `loo`	R
cmdstanpy	Python wrapper around CmdStan; returns `CmdStanMCMC`	Calls `cmdstan`; used with `arviz`	Python
rstan	R interface that embeds Stan in-process; returns `stanfit`	Alternative to CmdStan; used by `rstanarm` and `rstantools` packages	R

2.2.3 Workflow, modeling, and packaging

Package	Description	Relations	Language
brms	Flexible multilevel / distributional regression via formulas; returns `brmsfit`	Generates Stan; backends `cmdstanr` / `rstan`; used with `loo`, `bayesplot`	R
rstanarm	Precompiled applied regression models; returns `stanreg`	Built on `rstan`; used with `loo`, `bayesplot`, `projpred`	R
posterior	Tools to manipulate and summarize draws (R-hat, ESS, etc.)	Shared draws API for `cmdstanr` and analysis packages	R
bayesplot	ggplot2 plots for MCMC diagnostics, PPC, and posteriors	Uses draws from interfaces / `posterior`	R

2.2.4 Workflow, modeling, and packaging (continued I)

Package	Description	Relations	Language
loo	Approximate LOO-CV (PSIS-LOO), WAIC, and model weights	Needs pointwise `log_lik` from fitted models	R

Package	Description	Relations	Language
arviz	Diagnostics, plots, and model comparison in Python; returns `xr.DataTree`	Python counterpart to `posterior` / `bayesplot` / parts of `loo`	Python
priorsense	Prior and likelihood sensitivity analysis (power-scaling)	Uses `posterior`; related to PSIS methods in `loo`	R
projpred	Projection predictive variable selection	Uses `stanreg` (`rstanarm`) or `brmsfit` (`brms`) as reference	R

2.2.5 Workflow, modeling, and packaging (continued II)

Package	Description	Relations	Language
shinystan	Interactive Shiny GUI for MCMC diagnostics	Uses `stanfit` / `stanreg`; overlaps with `bayesplot`	R
rstantools	Tools to ship Stan models inside R packages; models typically return `stanfit`	Scaffolding used by packages such as `rstanarm`	R
posteriordb	Database of models, data, and reference posteriors	Shared resource for testing, teaching, and CI	R / Python / data

3 How to contribute to Stan?

By the end of this section, you will know the different ways of contributing to Stan, from engaging on Stan Discourse, over filing bug reports and feature requests, writing documentation and case studies, to contributing code. You will know how to pick a contribution path that fits you, write a clear GitHub issue or pull request, and follow project norms.

> **!** Important
>
> You don't need to be a professional software developer or statistician in order to contribute to Stan!

3.1 Types of contributions

There are many ways to contribute to Stan, including:

- asking and answering questions on Stan Discourse
- reporting problems when interacting with the software or documentation
- requesting new features or improvements
- providing feedback on documentation
- contributing with a case study of your specific research field
- contributing code

3.2 Typical workflows for contributing

3.2.1 Participate in the Stan Discourse

- Stan Discourse is the main forum for users and developers to ask questions, discuss issues, and share ideas.
- We aim to keep this a friendly and welcoming space, guided by our:
  - Code of Conduct
  - Discourse guidelines

- We also discuss topics on Slack, but we encourage users to ask questions on Stan Discourse instead as it's open to everyone, so answers can help other community members too.

3.2.2 Feature requests and bug reports

- Go to the GitHub repository you want to contribute to.
- Get familiar with its Issues and PRs (e.g., loo).
- If you're a new contributor:
  - Look for labels like "help wanted" or "good first issue."
  - Avoid issues that involve refactoring or larger changes.
  - When in doubt, ask in the issue whether it's a good one to work on.
- To open a new Issue:
  - First check whether it already exists, to avoid duplicates.
  - Some tips for writing good Issues...

Writing good Issues: Bug Report

- Title: One sentence, specific, searchable.
- Body
  - What happened vs what you expected
  - Minimal reproducible example (i.e., small model/script/data that triggers the problem)
  - (Optional) Environment: OS, hardware if relevant
  - Exact steps to reproduce the problem.
  - Logs / error output (if possible as text in a code block, not a screenshot, so it's searchable)
- Examples:
  - bayesplot
  - brms

Writing good Issues: Feature Request

- Title: One sentence, specific, searchable.
- Body
  - describe the problem or limitation
  - (optional) propose solution
  - (optional) note scope of Issue: small, self-contained or does it open a design discussion?
- Examples:
  - bayesplot

Additional references:

- Mozilla's guide to writing good bug reports
- "How to Report Bugs Effectively" by Simon Tatham
- R package "reprex" (tidyverse): runs your code, captures output, and formats it (code + results) as ready-to-paste Markdown for GitHub issues

---

3.2.3 Feedback on documentation

- Clarifying confusing sections
- Adding examples or code snippets
- Fixing inaccuracies (e.g., when current behavior deviates from documented behavior)
- Flagging content that is missing or unclear
- Fixing typos, grammar, and broken links

3.2.4 Contributing a case study or tutorial

- list of case studies in Stan
- blog post by Bob Carpenter on expanding the Stan Users Guide

Checklist for contributing a case study

1. **Initial contact**
   - post case study or tutorial idea in the Stan Discourse first
2. **Content requirements**
   - write it as a narrative document, not just code

- use a reproducible-notebook format
- note exact software and compiler versions used
- fix random seeds so results are exactly reproducible

Checklist for contributing a case study (continued)

3. Licensing

- license code under an open source license (e.g., BSD or GPL)
- license text as Creative Commons (CC-BY is most common)
- author retains copyright (but Stan needs to be permitted to host and redistribute)

4. Metadata for published case study

- Title and Author(s) (optional affiliation)
- Short abstract/description (in 2-4 sentences, what problem is solved and why it matters)
- Keywords (3-6 terms for discoverability)
- Source repository link (e.g., a public GitHub repo containing the actual notebook/knitr source, separate from the rendered HTML)
- Dependencies list (package/library versions used)
- Rendered HTML version (the actual output others will read)

3.2.5 Contributing Code

Some packages have templates others don't, but overall a good PR description should incorporate the following information:

Writing a good PR description

Before you start:

- Have you read the contributing guidelines and AI Contribution Policy?
- Contributing to Stan core libraries (e.g., `stan`, `math`, `stanc3`)? Checkout Developer Wiki?
- Is your PR small and focused on one logical change?
- Not ready for review yet? Open it as a **Draft PR** so reviewers know to hold off.

Title: One sentence, specific, searchable.

- Good: `Fix incorrect ESS calculation for thinned chains`

- Avoid: `Bug fix`

Writing a good PR description (continued I)

Body:

- Link PR to Issue via GitHub keywords like `Fixes #123` or `Closes #123`

Description

- Describe what has been changed and why.
- Be concise and use your own words (try to avoid using AI-generated text).
- Add any uncertainties or open questions you have here as well.

Breaking changes: Does this PR change existing behavior, APIs, or output? If yes, describe the impact.
Tests: (if applicable) Which tests have you added?
Documentation / Vignettes: (if applicable) Have you added documentation?

Writing a good PR description (continued II)

Checklist

- Style is consistent with existing code and docs
- Documentation or vignette renders locally
- Tests and `devtools::check()` pass
- `NEWS.md` entry added
- If you used an AI assistant, please disclose and review changes critically
- For the full checklist, see checklist

TODOs: List any remaining tasks you need to complete before the PR is ready to be merged.
Examples:

- bayesplot
- brms

3.2.6 Some reflections on the use of AI-assisted development

- Stan AI Contribution Guideline
- Use your own words when writing the PR description and comments in a PR review process
- Review all changes made by an AI assistant critically and ask yourself:
  - Is this change correct?
  - Do I need this change?
  - Is this change in the scope of the PR?
- When in doubt about something, ask rather the Stan developers than the AI assistant

4 Where to find information?

By the end of this section, you'll have a good sense of the main Stan resources including user guides, tutorials, case studies, and API documentation. We'll walk through how to match your question to the right resource.

4.1 Different layers of documentation

The Diátaxis framework provides a useful way to think about the different types of content that make up documentation. It divides documentation into four categories:

Type	Focus	Stan examples
Reference	What is this?	Function/package index, Stan math docs, Stan reference manual
How-to guides	How do I solve this?	Vignettes, case studies
Tutorials	How do I get started?	User's Guide walkthroughs, example-models
Explanation	Why is it like this?	Wiki, discussions on Discourse

The four are not a ranking, and no single document has to serve all of them. Most frustration with documentation comes from looking for one kind and finding another.

4.1.1 Reference ("What is this?")

Dry, complete, factual descriptions of the machinery. Structured for lookup rather than reading front to back.

- *"What is a valid Stan program? What are blocks, types, and constraints, and how do they behave?"*
  - see Stan reference manual
  - grammar/rules of the Stan language
  - analogous to a grammar book

- *"What does `normal_lpdf` in `stan_code` do, and what arguments/types does it expect?"*
  - see Functions reference
  - built-in vocabulary of the Stan language
  - analogous to a dictionary
- *"What does `loo()` in the loo R package do, and what arguments does it take?"*
  - see Package index (API) of corresponding downstream package
  - Examples: loo, ShinyStan
  - Analogous to a manual for a specific translation tool

4.1.2 How-to guides ("How do I solve this?")

Recipes for a specific task, aimed at someone who already knows the basics and has a goal in mind. In the Stan ecosystem these are mostly vignettes, published on each package's website.

- *"How do I fit a Stan model?"*
  - see Stan user guide
- *"How do I plot MCMC draws with bayesplot?"*
  - see Vignette in bayesplot
- *"How do I perform leave-one-out cross validation with loo?"*
  - see Vignette in loo
- *"How do I estimate multivariate models in brms?"*
  - see Vignette in brms

4.1.3 Tutorials / Case Studies ("How do I get started?")

Guided lessons that take a newcomer through a complete, working example. The goal is not to solve your problem but to build familiarity. The path is chosen for you and is meant to succeed.

- *"I have never used Stan. Where do I start?"*
  - see Tutorials
- *"How do these pieces fit into a full Bayesian workflow?"*
  - see case studies on the Stan website, in the Bayesian Workflow book, and on Aki Vehtari's website

4.1.4 Explanation ("Why is it like this?")

Context, design decisions, and alternatives considered. This is the material you read away from the keyboard, and the only category where opinion and history belong.

- *"What is the reasoning behind a method, and how was it validated?"*
  - see the paper accompanying the package, e.g. posterior, brms, bayesplot, ArviZ
- *"Why is Stan designed this way?"*
  - see Discourse discussions
  - see Wiki

4.1.5 Note about design review

https://github.com/stan-dev/design-docs

5 Checklist

5.1 Contributing to R packages

Getting started

- Check open issues or open a new one to discuss your proposed change before starting significant work (for bigger changes; typo/doc fixes can skip this)
- Check the repository for a `CONTRIBUTING.md` with package-specific guidance

AI contribution policy

- if you used AI tools (e.g. Claude, ChatGPT, GitHub Copilot) to help write your contribution, review the Stan project's AI Contribution Policy before opening a PR
- disclose AI assistance in your PR description if the policy requires it
- you remain responsible for understanding, testing, and being able to explain any AI-generated code you submit

Fork and clone

- raw git: fork on GitHub, then `git clone https://github.com/your-username/repository-name.git`
- usethis: `usethis::create_from_github("stan-dev/repository-name", fork = TRUE)`

Create a branch for your changes

- raw git: start from the main/master branch: `git checkout -b fix/123-short-description`
- usethis: `usethis::pr_init("brief-description-of-change")`

Make your changes on this branch

- keep your branch up to date with upstream
- install development dependencies `devtools::install_dev_deps()`
- debug with `devtools::load_all()` and `debug(new_function)`

- if you add a new function:
  - add tests and run them with `testthat::test_file()` or `testthat::test_dir()`
  - add documentation to new functions and examples
  - render the documentation with `devtools::document()`
- if you add a new vignette:
  - ensure vignette is rendered locally with `devtools::build_vignettes()`
- ensure you follow the style of the existing code/docs

i Contributing or changing plots (bayesplot in particular)

- bayesplot uses `vdiffr` for visual regression testing. In visual regression testing plots are compared against reproducible SVG snapshots rather than just checked for errors
- for any new plotting function or a change to an existing plot's appearance, add a graphical test using `vdiffr::expect_doppelganger("descriptive-title", plot_object)` in the relevant test file
- run `devtools::test()`, new or changed snapshots will be flagged
- review flagged snapshots with `testthat::snapshot_review()`, which opens an interactive Shiny app showing old vs. new SVG side by side
- only accept a snapshot change if the visual difference is the one you intended

i Update NEWS.md

- add a bullet for your change, placed just below the first header in NEWS.md
- follow the existing formatting and tone used in that file (check recent entries for the convention)

i Finalize your changes

- Add and commit your changes in reasonable chunks `git add <files>` and `git commit -m "short description of your changes"`
- cheatsheet for conventional git commit types
- run the full test suite with `devtools::test()`
- run the full check with `devtools::check()`
- push your branch
  - raw git: `git push origin fix/123-short-description`
  - usethis: `usethis::pr_push()` (opens the PR flow in your browser for you)

Open a PR

- add a detailed description of your changes (see How to write a good PR description)
- expect CI to run additional checks automatically